Chapter 81
Extension of the Dynamic Geometry Software for CeDG Support and Application to the Accurate Flattening of Developable Spatial Surfaces

Manuel Prado-Velasco and Laura García-Ruesgas

Abstract CeDG is a computer-based approach to build parametric 3D models of systems based on Descriptive Geometry (GD) procedures. The CeDG version used in previous studies was based on a raw version of GeoGebra (dynamic geometry software), which did not implement new tools, and particularly commands to measure lengths in locus based curves. This limitation prevented the complete implementation of the method to calculate the true flat pattern of oblique conical surfaces, forcing the use of approximation methods. In this study, we have extended GeoGebra with a set of new commands—tools, including several focused to the computation of lengths along locus—based curves. The flat pattern of a generic oblique cone has been computed using these new tools, as a case study to assess the accuracy and computational efficiency of the Geogebra extension for CeDG. Although the system was flattening with minor errors (0.4–1.3%), and the extension of GeoGebra supports the reliability and functionality of the new tools, the computation of locus lengths induced some ripple and loss of smoothness in the dynamic response of the system. Our findings suggest that the GeoGebraScript code and tools—command techniques used in CeDG evolution should be combined with Java/C GeoGebra source additions to reach a better computer efficiency in CeDG.

Keywords Computer extended descriptive geometry (CeDG) · Computer graphic parametric modelling · Dynamic geometry software · GeoGebraScript · Locus

M. Prado-Velasco (✉) · L. García-Ruesgas
Multilevel Modelling and Emerging Technologies in Bioengineering Group and Graphics Engineering Department, Universidad de Sevilla, Sevilla, Spain
e-mail: mpradov@us.es

C. Manchado del Val et al. (eds.), *Advances on Mechanics, Design Engineering and Manufacturing V*, Lecture Notes in Mechanical Engineering,
https://doi.org/10.1007/978-3-031-72829-7_81

81.1 Introduction

The Computer extended Descriptive Geometry (CeDG) approach has been devised as a complementary tool to current CAD technology for the analysis and design of 3D geometric systems. It provides the capability to address spatial analysis problems inherent to Descriptive Geometry by integrating into computational models of 3D geometry [1]. CeDG technique combines the problem-solving prowess of descriptive geometry methodologies with the proficiency of dynamic geometry tools, enabling the implementation of graphic-mathematical models with dynamic parameterization [2].

CeDG facilitates the analysis of the dynamic behaviour of geometric constructions while preserving their integrity. Several studies have demonstrated the potential of the CeDG technique in advancing sheet metal fabrication, surpassing certain constraints of current CAD systems [3]. Moreover, it has proven valuable in modelling mechanical systems, where geometric parameters are intricately tied to functional parameters through algebraic relationships. Integration of these relationships into the CeDG model streamlines the acquisition of design values throughout the modelling process.

A CeDG model describes a three-dimensional geometric system through a set of mathematical entities organized according to a construction sequence, maintaining dependency relationships with preceding entities in said sequence. Each mathematical entity is defined by means of an algebraic object that may be associated with a graphic object. In general terms, algebraic expressions can be defined by parameters, which are dynamically controlled and allow for dynamic modification of the model. An important aspect of a CeDG model lies in preserving the integrity of the curves and surfaces defining the 3D system. Unlike CAD systems, these are not substituted by approximations based on B-splines or similar methods unless explicitly required [4].

As discussed in a seminal study [1], the success and widespread adoption of CAD over recent decades have relegated descriptive geometry to a significantly reduced domain in academic education [5]. Although there exist some contributions that try to recover the DG as a problem-solving methodology implemented in computers, as the CADOP approach recently published [6], to the best of our knowledge, CeDG technique is the first methodology based on descriptive geometry for 3D computer modelling with a focus on both academic and professional fields.

Because of the widespread usage and the evolution of GeoGebra since the beginning of 2000s, previous studies implemented the CeDG approach into GeoGebra as the required Dynamic Geometry Software (DGS). However, GeoGebra was designed as an open-source system for learning mathematics and geometry, and although the current version contains an advanced 3D module that can export models to STL for rapid prototyping by 3D printing, there are some weaknesses that make it difficult to adopt it as a professional software in the technology field.

According to the exposed needs, the *first objective* of this study was to implement a first extension of GeoGebra functionalities using the technique of building new tools—commands, interactive controls and GeoGebraScript.

In addition, because of the relevance of the locus object to compute surface intersections and flattening of developable surfaces in CeDG, and due to the lack of a command for calculating lengths along 2D locus, the *second objective* of this study was the inclusion of a set of tools (commands) to overcome this limitation.

The feasibility of GeoGebra to be extended according to this methodology, and the accuracy and computer efficiency of these tools, and particularly those involved in the computation of locus lengths, will be assessed. A case study concerning the flattening of an oblique cone to obtain a dynamic parametric model of the flat pattern, computed through the method previously published [7] will be presented. The algorithm for flattening a developable surface is concisely described in the following Section. More details may be found in previous publications [7, 8].

81.2 An Algorithm for Flattening a Developable Surface

Referring to Ω as the flat cylindric section exerted by a plane perpendicular to a general cylinder, the conservation of lengths, as well as the keeping of the angle between any generatrix and the tangent to Ω through the point intersection generatrix—tangent, lead to write the flat transformed, τ of any curve C on the cylinder as [8]:

$$L(\omega) \equiv \tau = locus(P_\tau(f_D(\omega)), \omega \in \Omega) \tag{81.1}$$

In this equation, the function $L(\omega)$ represents a parametric formulation of the curve τ, using $\omega \in \Omega$ as a parameter.

As it will be shown in the example on this Section, the transformed P_τ is exactly computed thanks to the fact that the flat transformed of Ω is the straight line, D. Therefore, a generatrix of the ruled surface, $g(\omega)$, transform onto $g_D(d) = g_D(f_D(\omega))$, which is also a straight line, that is perpendicular to D. The function $f_D(\omega \in \Omega) = d \in D$, is a mapping function that transforms a point from Ω onto a point in D.

Despite Eq. (81.1) seems complicated to compute, it is completely and implicitly determined from the algebraic entities (dynamic objects) of the CeDG model created when calculating the transformed point P_τ from a generic point P_C on C, using a generatrix $g(\omega \in \Omega)$ that contains P_C.

Equation (81.1) can be equivalently expressed when the point in Ω is controlled by a real parameter t. Therefore, Eq. (81.1) can be expressed in a more general form as:

$$L(\omega) \equiv \tau = locus(P_\tau(f_D(\omega(t))), t) \tag{81.2}$$

The use of this algorithm is demonstrated in the example presented below, where the exact flat pattern of a cylindrical graft, a common application of ruled surfaces, is obtained. As depicted in Fig. 81.1, the initial data consists of a straight cylindrical graft, which intersects with a main duct defined by an oblique cylinder with circular

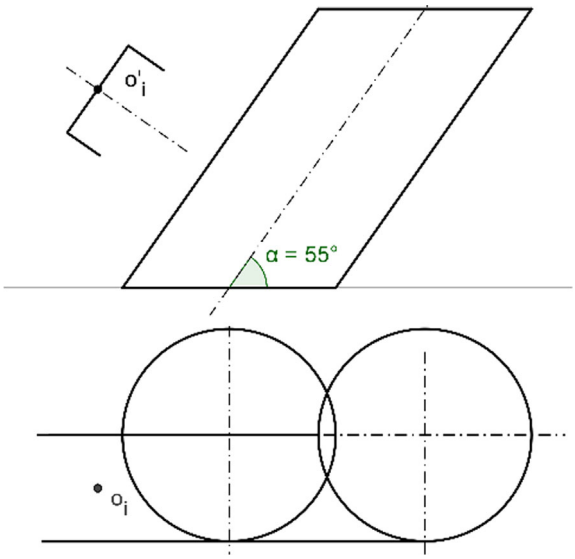

Fig. 81.1 Cylindrical graft to be inserted on the main duct

sections. The angle of the main duct's axis relative to its base is α. The vertical projection of the circular mouth of the grafted cylinder, with center at O_i, is provided. The model is completely parameterized and Fig. 81.1 shows its dimensions for a convenient selection of values that simplifies the building. The relative position of the cylinders is fixed by a common frontal plane tangent to both cylinders.

The radius of graft cylinder is lesser or equal than that of the main duct, to ensure that the graft penetrates the main duct, and $\alpha \in (0°, 90°)$ to ensure the encounter.

The obtention of the flat development of both ducts requires calculating the intersection curve between both cylinders beforehand. To do this, the free point p has been defined on the horizontal projection of the circular opening of the grafting cylinder, as can be seen in Fig. 81.2. Point p defines a frontal plane that generates cutting the generatrices in both cylinders. The upper generatrix of the grafted cylinder (containing p') is presented, alongside the two cutting generatrices of the main duct. The first generatrix intersects with that of the grafted cylinder at P_1, and the second generatrix intersects at P_2. Only P_1 belongs to the entrance orifice of the graft.

The resulting intersection curve is represented by the projections $locus(p_1, p)$ for the horizontal projection and $locus\left(p'_1, p\right)$ for the vertical projection. Both projection curves agree with the following equation [7]:

$$L(\omega) \equiv \sigma = locus(p_\sigma(p), p \in \mathcal{B}) \tag{81.3}$$

where σ is the projection curve of the intersection curve onto a plane of interest.

The main duct is flattened using the algorithm according to Eq. (81.2), as depicted in Fig. 81.3. The flat pattern consists of the transformations of the base and lid flat sections and the transform of the connecting warped curve. Ω is defined as the

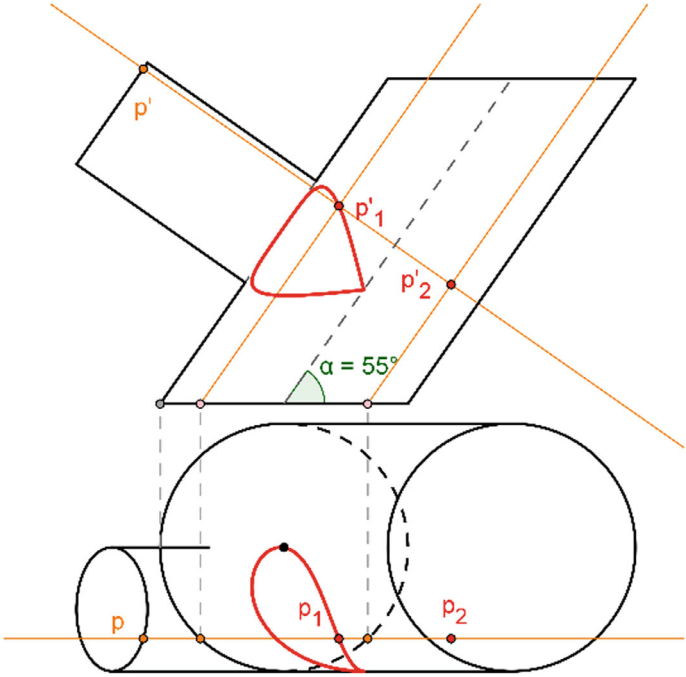

Fig. 81.2 Projections of the main duct with the cylindric graft

intersection of the cylinder with the plane perpendicular through point C. A generatrix containing point B has been taken as a reference for the development. This generatrix is contained in the furthest frontal plane from the vertical projection plane. The real magnitude of the Ω has been calculated by applying a rotation that places it on the H′ horizontal plane.

Using the free point (Q) on the real magnitude of Ω, the generatrix containing it allows obtaining its projections q and q′. The distances from q′ to base $\left(q'_b\right)$ and lid $\left(q'_t\right)$ are in real magnitude. Placing Q on the flat domain at the distance of the elliptical arc (B)–(Q) from B, the transformed generatrix can be drawn, and the points Q_b and Q_t can be located.

The flatten base and lid curves are obtained through their parametric functions according see Eq. (81.1): $locus(Q_b, (Q))$ for the base and $locus(Q_t, (Q))$ for the lid. The encounter is flatten using p as t parameter in Eq. (81.2), which gives $locus(P_1, p)$. Figure 81.3 shows the construction of generating points using standard techniques in descriptive geometry.

In the case of a generic cone, it is necessary to have a surface that intersects with the cone in a curve that may be used as support curve, A, for generatrix lines in the flat domain. This is the case of the sphere, because of its flattened state is a circular arch with radius equal to that of the sphere [7]. Using any planar section to the cone as set Ω to define the parameter $\omega \in \Omega$, the flat transformed, τ of any curve C on the

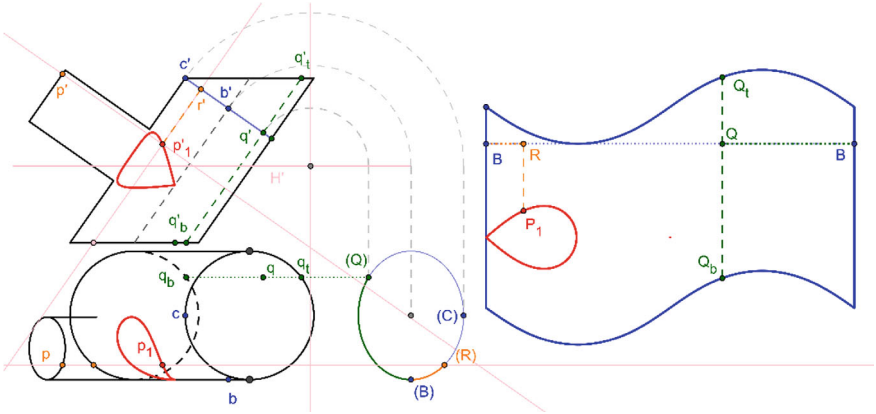

Fig. 81.3 Flat pattern of the main duct (right) and projections of the system (left)

cone will be given by the equation:

$$L(\omega) \equiv \tau = locus(P_\tau(f_A(\omega)), \omega \in \Omega) \tag{81.4}$$

where the mapping function that transform a generatrix of the ruled surface, $g(\omega)$, to $g_D(f_A(\omega))$ in the flat domain requires to flatten the support curve, A, with the aim of measuring distances between points onto it. As indicated in previous studies, GeoGebra does not implement a built-in command that gives the length between points along a locus—based curve [8], what limits the use of this methodology in the case of oblique cones. The following sections show several tools that have been implemented under GeoGebraScript code to compute this length.

81.3 Set of New GeoGebra Tools for CeDG

As a first stage towards the development of a new branch of GeoGebra software focused on the analysis of tridimensional geometric systems (GeoCeDG), we have added a set of tools that facilitate the execution of several descriptive geometry procedures and frequent geometric constructions. These were constructed by encapsulating the graphical building that provides the expected results, as dynamic objects that depend on input dynamic objects.

The following Fig. 81.4 shows some of these tools grouped by categories as dropdown menus, command-based calculations of the length of several curves (linear, slope, and conical curves), procedures for implementing several frequent structures, and tools that facilitate coordinate and distance transportation from left to right menus.

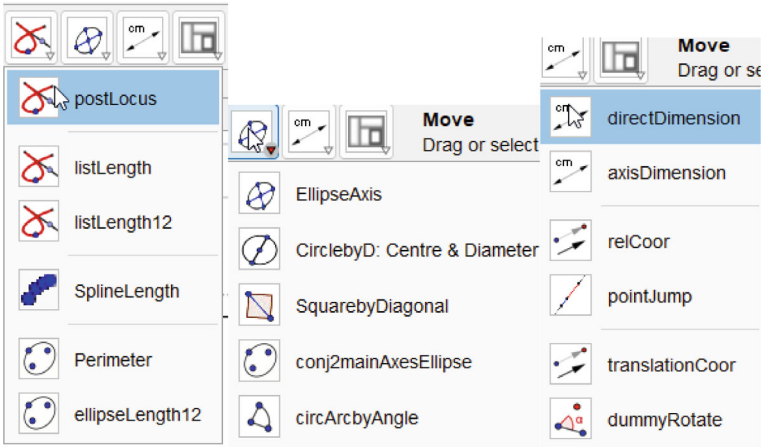

Fig. 81.4 Drop–down menus of several specific tools implemented in GeoCeDG, organized in categories: length computations (left), 2D Geometry building (middle) and coordinates and distances measuring and transportation (right)

We have verified the capability of this encapsulation technique of GeoGebra as a powerful methodology to build new algebraic procedures, avoiding even the need to write them through code. The implicit construction may include conditional and iterative objects to analyze the inputs and give more general solutions. For example, when building ellipses from their main axis, the tool can detect which is the largest to find the focus points on the ellipse. Furthermore, the largest axis can vary because of changes in model parameters, thereby enhancing their validity.

The following Fig. 81.5 shows a construction that gives the major axes of the ellipse starting with a couple of conjugated axes (`conj2mainAxesEllipse`). Later ones are defined through the input points `A, B, C` (marked in orange). As the technique depends on the largest conjugated axis, this feature must be detected by code.

The code (GeoGebraScript) of this construction is shown in the following Fig. 81.6. According to it, the radius `RM` of circle `e` with center `A` is defined using the largest conjugated axis, detected by the conditional instruction `RM = If(se1 > se2, se1, se2)`.

The auxiliary circle `k`, is built using as center the middle point between the smallest conjugated axis extreme, `B`, and the intersection of the perpendicular through the largest conjugated axis with circle `e`. Again, several conditional instructions allow automating the selection of the compliant objects to finally obtain the major axes, `i` and `j`, the half lengths of them, `p` and `q`, and the end points of major axes, `L, K, N`, and `M`.

In addition to the implementation of new tools, we have included other functions to manage the scales, to format the canvas according to ISO 216 sizes, and automate the calling to commands that lack associated tools (as is the free copy of objects).

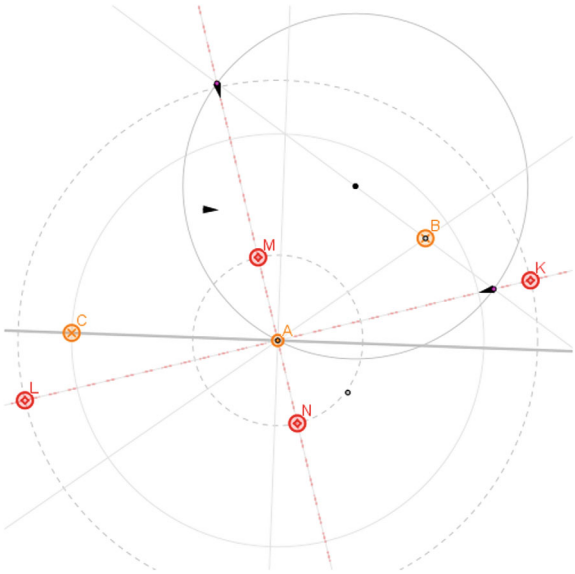

Fig. 81.5 Determining of major axes of the ellipse using two conjugated axes as inputs. Selected points indicate inputs (orange) and outputs (red) of tool, and match with selections in the associated code (see next figure)

Figure 81.7 shows the controls associated with the printing of ISO 216 formats, together with the extended tools bar.

Figure 81.8 shows the background sheet generated for a landscape A3 format with the new tool `sheetISOAnLand`. Final views may be exported to PDF and other raw and vectorial formats.

The computation of the length of a locus curve is performed with a 2-stage procedure.

First, the `postLocus` tool calculates a list with points that cover the locus curve (input argument). The covering is controlled by two filtering parameters, Nmod and epsDer. Second, the length of the locus curve is computed with `locusLenth` tool (whole locus) or `locusLength12` tool (from points 1 to 2), using the list of points as input. This technique was developed to avoid the need of converting the locus curve in an algebraic equation since this is a hard task. GeoGebra implements Giac algebra to compute the Gröbner bases as the symbolic strategy to obtain the locus equation [9], but practical solutions are strongly limited.

El commando `postLocus(locus, Nmod, epsDer)` executes the following GeoGebraScript code:

```
1 autoListPoint = First(locus, Length(locus))
2 nm = floor(Length(autoListPoint) / 2)
3 epsCont = Nmod UnitVector(Vector(autoListPoint(nm), autoList-
Point(nm + 1))) Vector(autoListPoint(nm), autoListPoint(nm + 1))
4 ListLeft = Sequence(If(epsDer Length(Vector(autoListPoint(n +
1), autoListPoint(n + 2))) ≤ UnitVector(Vector(autoListPoint(n),
autoListPoint(n + 1))) Vector(autoListPoint(n + 1), autoList-
Point(n + 2)) < epsilonCont, autoListPoint(n + 2), (-1)^1.5), n, nm,
Length(autoListPoint) - 2)
```

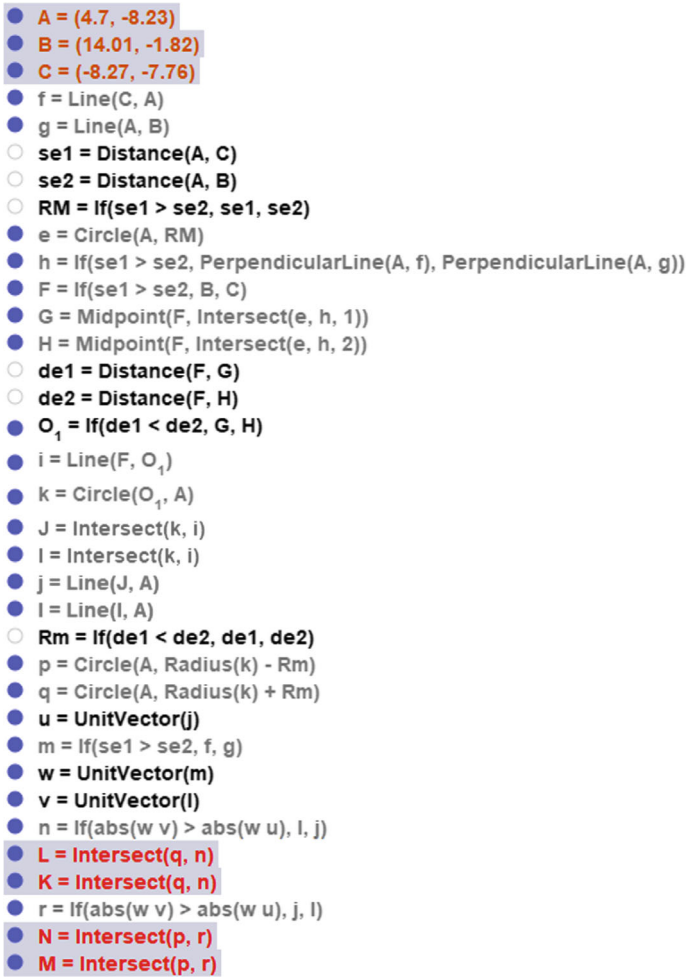

Fig. 81.6 GeoGebraScript source associated with the graphical building of `conj2mainAxesEllipse` tool/command. The input objects are selected and colored in orange (A, B, C), whereas the output objects are selected and colored in red (L, K, M, N). All of them are labeled in the graphical construction of the previous figure

```
5 ListRight = Sequence(If(epsDer Length(Vector(autoListPoint(n -
1), autoListPoint(n - 2))) ≤ UnitVector(Vector(autoListPoint(n),
autoListPoint(n - 1))) Vector(autoListPoint(n - 1), autoList-
Point(n - 2)) < epsilonCont, autoListPoint(n - 2), (-1)^1.5), n, nm
+ 1, 3, -1)
6 ListFilt = Join({Reverse(List1), {autoListPoint(nm), autoList-
Point(nm + 1)}, List2})
7 ListFiltFinal = RemoveUndefined(ListCorr)
```

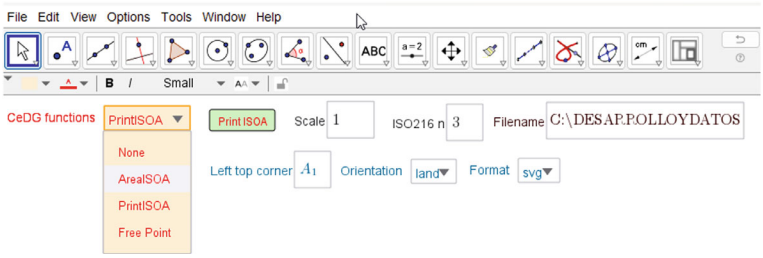

Fig. 81.7 Drop-down list that controls the conditional visibility of interactive controls (buttons, edit boxes, and dependent drop-down lists)

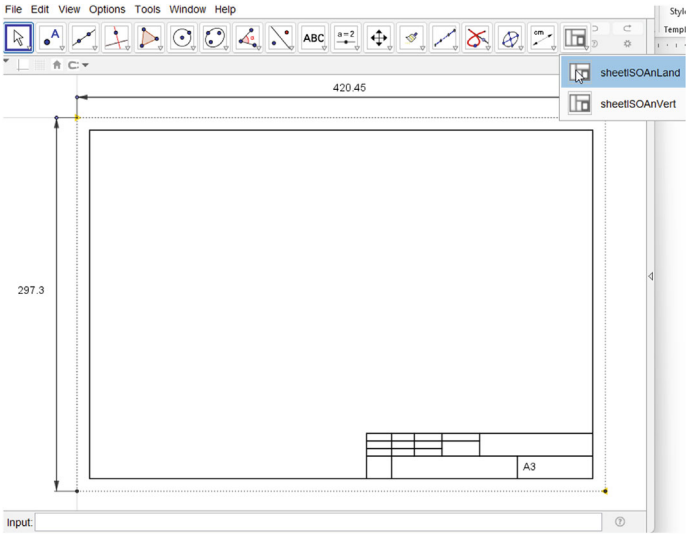

Fig. 81.8 Main 2D window with a landscape A3 format, generated through the new tool `sheetISOAnLand`, which also controls the drawing scale

After extracting the whole points that covering the locus curve given as input (line 1), the total number of points are saved in $2 \cdot$ `nm`, and `epsCont` is defined as Nmod times the norm of vector defined from point `nm` to point `nm + 1`, where Nmod is a metric input number >1 (line 3).

`ListLeft` and `ListRight` are list of points at left and right of the middle point nm, extracted from the original autoListPoint if the following condition is fulfilled (lines 4 and 5):

$$\text{epsDer}|\text{Vector(i)}| < \text{Unitvector(i-1)Vector(i)} < \text{epsCont} \qquad (81.5)$$

where epsDer is an input value < 1, and Vector(i) is the vector from point i-1 to point i from autoListPoint list and ordered from the middle to the first point (ListLeft) or from the middle to the end point (ListRight).

The Eq. (81.5) guarantees that the distance between point i and previous (i-1) is not excessive (< epsCont) to avoid discontinuity, and that the direction of vector from i-1 to i, (Vector(i)) keeps the direction of previous vector (> epsDer |Vector(i)|) to avoid lack of derivability. This condition applies if points cover properly the locus curve (gaps between points are small enough), for differentiable curves. The value (-1)^1.5 is not defined, what marks the outlier that is finally removed by the `RemoveUndefined` command (line 7).

The postLocus filtering is needed to detect and remove some outlier points provided by Locus command in GeoGebra.

The length of the whole locus is computed through locusLength, which gives the algebraic object: `Sum(Sequence(Distance(Element(listTarget, i), Element(listTarget, i + 1)), i, 1, Length(listTarget) - 1))`. This approximates the length of the locus curve as the distance between points that cover properly and smoothly the curve.

The tool `listLength12` gives the length between two points A and B, when this is needed. This tool uses the same approach as `locusLength`, but previously must compute the index iA that minimizes the distance between ListPoints(iA) and point A (first point), and similarly for index iB and point B (second point). This is performed as follows:

```
Sequence(Vector(listTarget(i), A) Vector(listTarget(i + 1), A) <
epsilon, i, 1, Length(listTarget) - 1)
indA = IndexOf(true, verifyepsA)
```

The concept is that the scalar product between two consecutive vectors from A to the point listTarget(i) and to the point listTarget(i + 1) is always positive, except when one of the points is A or A is just between them. As point A may be near the curve, the input metric `epsilon` allows controlling the accuracy of the minimization. Once the points have been found, the length between them is computed as:

```
length = If(indA < indB, Sum(Sequence(Distance(Element(listTarget,
i), Element(listTarget, i + 1)), i, indA, indB -
1)), Sum(Sequence(Distance(Element(listTarget, i),
Element(listTarget, i + 1)), i, indA, Length(listTarget)
- 1)) + Sum(Sequence(Distance(Element(listTarget, i),
Element(listTarget, i + 1)), i, 1, indB - 1)))
```

We have applied these new tools to implement the Eq. (81.4) in the process of flattening an oblique cone in CeDG in the next Section.

81.4 A Study of Oblique Cone Flattening

The following Fig. 81.9 shows an oblique cone implemented in CeDG, intersected with a sphere (yellow) with center in cone vertex (v′–v) and radius equal to the length of generatrix V−R. It is presented only the vertical projection of the sphere for convenience.

The encounter cone—sphere has been computed starting with the point x on the cone base, whose generatrix meets the sphere at P_x point. Once the projections of this point are determined, the intersection curve, S, is calculated as $locus\left(p'_x, x\right)$ and $locus(p_x, x)$ for vertical and horizontal projections, respectively.

With the aim of measuring distances along S, which is the support curve for flattening the cone, a right cylinder, perpendicular to the horizontal plane and limited by curve S, is first flattened. This has been executed using the methodology for general cylinders, described in the algorithm Section.

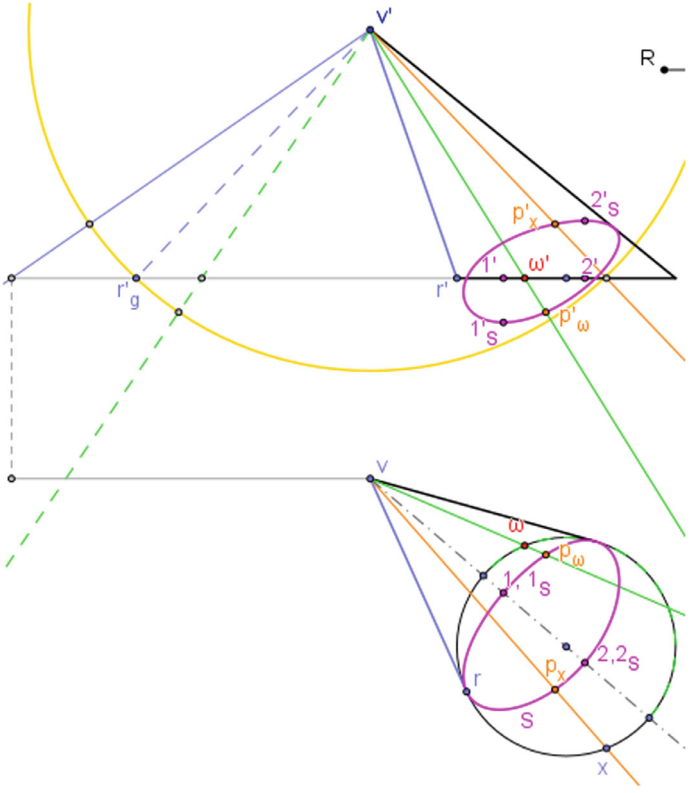

Fig. 81.9 Dihedral projections of cone, intersected with a sphere (yellow) with v'- v center to give the warped curve S (Support, pink projections). See text for more details

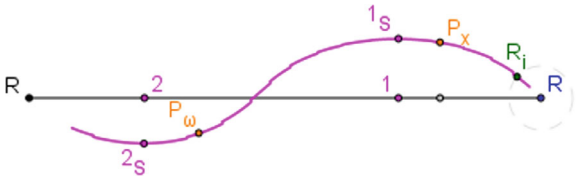

Fig. 81.10 Flat pattern of S as pertaining to the right cylinder. See text for details

Figure 81.10 shows the flat pattern of S curve according to Eq. (81.2) as $locus(P_x, t)$, where the x point is mapped with the real parameter t. We have compared the computer efficiency of $locus(P_x, t)$ against $locus(P_x, x)$ in GeoGebra, resulting in a slight improvement of this one for t version of the locus curve.

The distance between R and the position of the generatrix containing P_x in the base of the right cylinder was determined with l1 = `postLocus` ($locus(p_x, x)$, Nmod = 4, epsDer = 0.6), and `locusLength12`(l1, r, p_x. `epsilon` = 0.00001).

As shown in Fig. 81.10, the $locus(p_x, x)$ curve is not well covered in the proximity of R point, what impedes the computation of lengths in this zone, producing a bad flat pattern of S around R. We have reduced the definition interval of t to avoid the vicinity of R, what explains the lack of flat pattern of S near R.

Figure 81.11 shows the half flat pattern of the cone obtained through Eq. (81.4), where the $f_A(\omega)$ mapping function is calculated through the lengths along the A support curve (S in our example), which is kept during the flattening process of S as pertaining to the right cylinder (Fig. 81.10). The symmetry of the cone with respect to the plane perpendicular to the horizontal plane through V-2 allows computing only half of the pattern farthest of the point R.

The half flat pattern has been computed as $locus(\omega_F, \omega)$ where ω has been limited to the half circular base that does not contain r (Fig. 81.9). The distance along flattened support curve between P_ω and R_i (Fig. 81.11) was calculated using the flat pattern of Fig. 81.10, with l2 = `postLocus`($locus(P_x, t)$, Nmod = 7, epsDer = 0.1), and `locusLength12`(l2, R_i, P_ω `epsilon` = 0.001). Once P_ω is placed on the flattened support curve, its generatrix and ω_F can be computed, what solves implicitly the function $f_A(\omega)$.

The accuracy of the computed pattern is determined in Table 81.1, comparing the length of the flat pattern with the true length (half perimeter of the circular cone base). Values refer to a diameter of 60 mm. According to symmetry and derivability of the pattern, the angle of tangents to the pattern in the contour generatrices must be 90°.

Figure 81.12 is the flat pattern computed through a well-known pyramid discretization, using a number of generatrices equal to seven. The length of the flat curve was 93.6 mm (0.7%) for this standard method.

Although errors are small, the power of the flattening algorithm using a spherical support curve in CeDG is not exploited with the new tools for locus length that has been implemented and presented in this preliminary study.

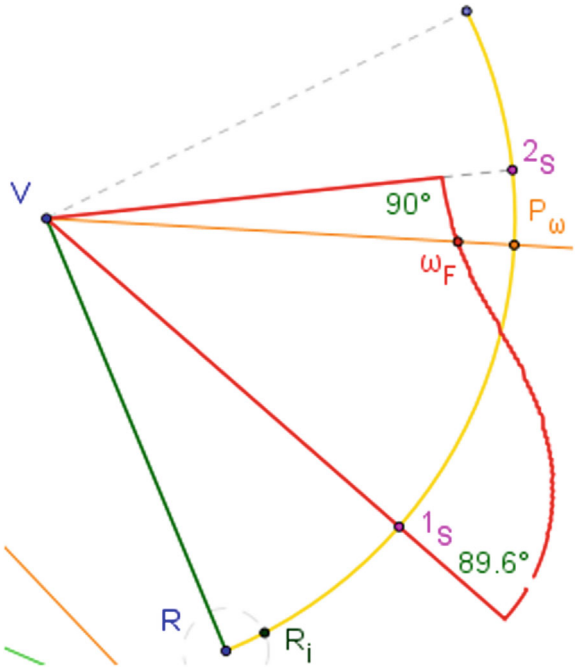

Fig. 81.11 Half flat pattern (red) of cone using parametric function obtained through sphere's-based algorithm with approximation of locus lengths. The flat pattern of support curve (yellow) as pertaining to cone is used to lead the location of cone generatrix

Table 81.1 Accuracy of selected metrics on flattening process due to locus length command

Dimension	True value	Computed value (error)
Length of flat curve	94.25 mm	95.04 (0.8%)
Angle (curve's tangent, V-2 generatrix)	90°	91.2° (1.3%)
Angle (curve's tangent, V-1 generatrix)	90°	89.6° (0.4%)

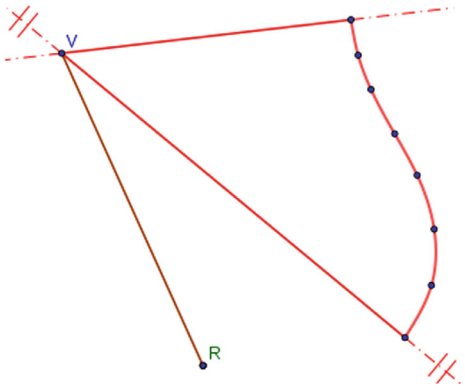

Fig. 81.12 Half flat pattern of cone using pyramid discretization. The number of generatrix lines is indicated with their dots on the flat pattern of the cone's circular base

81.5 Discussion and Conclusions

This study has assessed the capability of extending GeoGebra as a dynamic geometry software that supports the requirements of CeDG approach. As a preliminary stage, before starting the development of a new branch of GeoGebra based on Java/C++, this study has analyzed the limitations of the tools—commands that may be implemented in GeoGebra using graphical constructions and GeoGebraScript code.

According to the outcomes, the new tools may deliver dynamic objects that combine a very extensive set of functions and techniques, attending to diverse conditional requirements. However, this technology is not enough to modify existing commands, as occurs for the locus.

Locus commands provide a sophisticated mechanism that encapsulates the logic of a 3D geometry model (under CeDG perspective) to give a set of points that fulfils with complex laws. Although the implementation of locus has been improved in the latest versions of GeoGebra, there are still some limitations that must be corrected in CeDG.

One of them is the ability to control properly the covering of the locus domain according to the required accuracy, to allow the measurement of lengths along locus curves. Several techniques have been tested to perform a good filtering of some outliers presented in locus solution, but it is required a reformulation of the locus through the Java source to implement the requirements needed.

This is particularly relevant for the flattening algorithm tested in this study, where three locus objects are concatenated with the aim of computing the final flat pattern of the cone. Figure 81.13 shows the influence of filtering parameters in the covering of the last locus object (flat pattern of cone).

The top pattern shows that the covering seems proper (there are enough points sweeping the solution space), although the number of points reduce abruptly when nMod reduces from 10 to 2 and epsDer increases from 0 to 0.7. The final nMod penalizes non-homogeneous distributions of points, whereas epsDer does with change in the curve direction at small scales.

The reason for this behavior is best shown in Fig. 81.14, which enlarges the tiny 5 mm circle of Fig. 81.13.

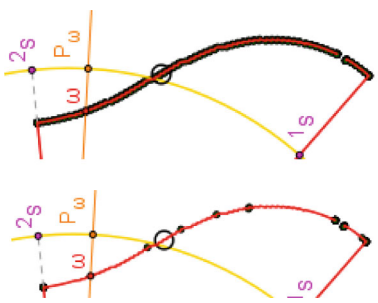

Fig. 81.13 Influence of filtering parameters in dots covering the locus-based curve domain to compute the length of the curve: top (nMod = 10, epsDer = 0), bottom (nMod = 2, epsDer = 0.7)

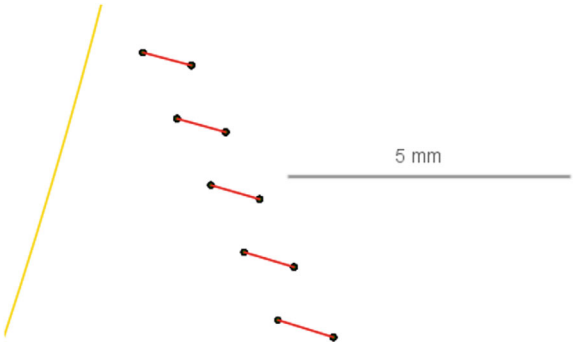

Fig. 81.14 Ripple in the dots covering the locus-based function due to accumulation of numerical errors in concatenated filtering. The flattened curve of Fig. 81.13 shows a circle of diameter 5 mm that corresponds to the present detail

The curve's ripple is associated with numerical errors that accumulate as a cubic power in this model. Accordingly, when filtering to penalizes change in direction, many of the points are rejected, resulting in Fig. 81.13—bottom.

Additionally, the computer load associated with `postLocus` and `locusLength` tools penalizes the smoothness of the dynamic response. This was due to concatenation of locus using filtering and optimization.

In summary, this study suggests that GeoGebraScript code together with the own methodology of GeoGebra for building tools allows the extension of the software to focus on computational modelling of 3D engineering systems, which is the objective of the CeDG approach. However, this methodology must be combined with the addition of changes in functions at java/C sources to extend the capability of locus dynamics objects, mainly. GeoGebra is an open-source software with GNU/GPL (mainly) license, which facilitates the later task.

Future work in CeDG will extend the locus objects, generating a new GeoGebra branch, called **GeoCeDG**.

References

1. Prado-Velasco M, Ortíz Marín R, García L, Rio-Cidoncha MGD (2021) Graphical modelling with computer extended descriptive geometry (CeDG): description and comparison with CAD. Comput-Aided Des Appl 18:272–284
2. Hohenwarter M, Fuchs K (2005) Combination of dynamic geometry, algebra and calculus in the software system GeoGebra. In: Computer algebra systems and dynamic geometry systems in mathematics teaching. Sarvari, Cs. Hrsg., Pecs, Hungary, pp 128–133
3. Prado-Velasco M, Ortiz-Marín R (2021) Comparison of computer extended descriptive geometry (CeDG) with CAD in the modeling of sheet metal patterns. Symmetry 13:685
4. Marsh D (2005) Applied geometry for computer graphics and CAD. Springer-Verlag, London, United States of America
5. Migliari R (2012) Descriptive geometry: from its past to its future. Nexus Netw J 14:555–571
6. Gutiérrez de Ravé E, Jiménez-Hornero FJ (2024) A 3D descriptive Geometry problem-solving methodology using CAD and orthographic projection. Symmetry 16:476
7. Prado-Velasco M, García-Ruesgas L (2023) Intersection and flattening of surfaces in 3D models through computer-extended descriptive geometry (CeDG). Symmetry 15:984

8. Prado-Velasco M, García-Ruesgas L, Ortiz-Marín R, Vázquez-López E (2023) Modelado paramétrico computacional de sistemas 3D mediante Geometría Descriptiva (CeDG): sistema diédrico. Editorial Aula Magna, McGraw-Hill Interamericana de España, Spain
9. Botana F, Kováks Z (2016) New tools in GeoGebra offering novel opportunities to teach loci and envelopes